Intelligence Artificiel
DM

ABARCA Elliot & POTEL Guillaume

Année universitaire : 2025 – 2026

Chargement du dictionnaire et lecture d'un fichier mail :

Pour réponde à la consigne de ne charger uniquement les mots d'au moins 3 lettres, nous avons opté pour une compréhension de liste en ne prenant que les mots supérieurs à cette taille.
Les mots parcourus initialement sont ceux lus dans le fichier donné en paramètre.
Voir figure 1.

```python
def charge_dico(fichierDico: str) -> list:
    """
    Fonction qui insert dans une liste `mots` tous les mots d'un fichier
    `fichierDico` dictionnaire (.txt) donné en paramètre.
    Les mots de moins de 3 lettres sont ignorés.

    Args:
        fichierDico (str)   : Le chemin vers le fichier dictionnaire.

    Returns:
        list                : La liste de tous les mots du dictionnaire.
    """
    f = open(fichierDico, "r")
    texte = f.read().upper()
    f.close()

    # On extrait les mots de 3 lettres au moins :
    mots = [mot for mot in re.findall(r'\b[A-Z]+\b', texte) if len(mot) >= TAILLE_MIN_MOT] # Mise en
MAJ pour insensibilité à la casse

    print("Chargé " + str(len(mots)) + " mots dans le dictionnaire")

    return mots[:-1]
```

Figure 1 : Code de la fonction charge_dico

Pour rendre insensible à la casse la lecture et recherche d'appartenance au dictionnaire, le dictionnaire est chargé en forçant les majuscules avec la fonction .upper().
On fait de même pour la lecture d'un fichier mail. On a d'ailleurs amélioré la lecture des mots d'un fichier mail en utilisant une regex, via la fonction .findall du module re, pour le filtrage de mot. Cela nous permet plus de précision puisqu'une simple séparation par le caractère espace nous produirait des mots ponctués qui ne serait pas reconnu dans le dictionnaire.
Pour retrouver un mot dans le dictionnaire, on utilise la fonction native à numpy .isin qui renvoie un tableau de booléen qui correspond exactement à la construction voulu de notre vecteur x : même taille que le dictionnaire, et booléen sur chaque mot pour signifier présence ou non.
Voir figure 2.

```python
def lireMail(fichierMail: str, dictionnaire: list) -> np.ndarray :
    """
    Fonction qui lit un fichier mail `fichierMail` et retourne un vecteur de booléens en fonction
    du dictionnaire.

    Ex.
        Si le mot de coordonné i dans le dictionnaire `dictionnaire` est présent dans `fichierMail`
        alors x[i] = True, sinon x[i] = False

    Args:
        fichierMail     (str)   : Nom du fichier mail à lire.
        dictionnaire    (list)  : Dictionnaire utilisé pour création du vecteur x.

    Returns:
        np.ndarray              : Le vecteur booléen d'appartenance
    """
    f = open(fichierMail, "r",encoding="ascii", errors="surrogateescape")

    # On extrait les mots du mail :
    texte = f.read().upper() # Mise en MAJ pour insensiblité à la casse
    f.close()
    mots = re.findall(r'\b[A-Z]+\b', texte) # cela ignore la ponctuation

    # Construction du vecteur :
    x = np.isin(dictionnaire, mots)

    return x
```

Figure 2 : Code de la fonction lireMail

Apprentissage :

Pour l'apprentissage des paramètres (correspond à la fréquence), on fait comme le sujet l'indique : parcours des fichiers (lecture) un par un.
On crée le vecteur b par addition de tous les x produit par la fonction lireMail. Cela est possible car x est un vecteur booléen et que les deux structures sont des vecteurs numpy.
Le résultat obtenu est donc un vecteur calculant le nombre d'apparition pour chaque mot j du dictionnaire. Il ne nous reste plus qu'à diviser par le nombre de fichier pour obtenir la fréquence, et grâce à numpy on le fait en une ligne.
Voir figure 3.
Pour apprendre ses paramètres, on le fait classiquement en appliquant la fonction sur les fichiers de la base d'apprentissage. On l'applique pour les SPAMs et pour les HAMs

Prédiction :

Comme on travaille sur des booléens, on simplifie les calculs avec x et not x (natif à numpy).
Pour éviter, comme précisé dans le sujet, les comparaisons sur des puissances, on utilise le log qui préserve les inégalités en permet de traiter les nombres correctement.
Voir figure 4.

```python
def apprendBinomial(dossier: str, fichiers: list, dictionnaire: list) -> np.ndarray:
    """
    Fonction d'apprentissage d'une loi binomiale à partir des fichiers d'un dossier.
    Retourne un vecteur b de paramètres qui, pour chaque j, calcul la fréquence d'apparition.

    Args:
        dossier         (str)   : Nom du dossier dans lequel piocher les fichiers de la base
d'apprentissage.
        fichiers        (list)  : Liste des fichiers d'apprentissage.
        dictionnaire    (list)  : Dictionnaire utilisé pour l'apprentissage.

    Returns:
        np.ndarray              : Vecteur de paramètres
    """
    N = len(fichiers) # On récupère le nombre total de fichier dans la base

    # Initialisation du vecteur b de même taille que le dictionnaire :
    b = np.zeros(len(dictionnaire))

    # b[j] = nombre de fichier contenant le mot dictionnaire[j] / nombre de fichier

    # Parcours des fichiers un à un comme le sujet le précise :
    for fichier in fichiers:
        # On accumule dans b les vecteurs (comme True = 1, on compte enft le nombre de fichier où
apparait chaque mot du dictionnaire)
        b += lireMail(dossier + '/' + fichier, dictionnaire)

    # on utilise epsilon pour eviter que log soit 0
    return (b + EPSILON) / (N + 2 * EPSILON)
```

Figure 3 : Code de la fonction apprendBinomial

```python
def prediction(x: np.ndarray, Pspam: float, Pham: float, bspam: np.ndarray, bham: np.ndarray) ->
tuple[bool, float, float]:
    """
    Fonction qui prédit si un mail représenté par un vecteur booléen `x` est un spam à partir
    du modèle de paramètres `Psam`, `Pham`, `bspam`et `bham`.

    Args:
        x           (np.ndarray)    : Vecteur booléen représentant un mail.
        Pspam       (float)         : Proba à priori SPAM
        Pham        (float)         : Proba à priori HAM
        bspam       (np.ndarray)    : Paramètres appris sur les SPAMs (fréquences)
        bham        (np.ndarray)    : Paramètres appris sur les HAMs (fréquences)

    Returns:
        tuple[bool, float, float]:  True si le mail donné en paramètre est classé comme SPAM par le
classifieur, False sinon.
                                    Renvoie aussi les proba calculées.
    """

    # Pour éviter les problèmes numériques, on passe aux logs :
    # On utilise aussi x et ~x (opérateur NOT numpy) puisque valeur booléennes
    log_spam = np.log(Pspam) + np.sum(x * np.log(bspam) + ~x * np.log(1 - bspam))
    log_ham  = np.log(Pham)  + np.sum(x * np.log(bham)  + ~x * np.log(1 - bham))

    max_log = max(log_spam, log_ham)

    prob_spam = np.exp(log_spam - max_log) / (np.exp(log_spam - max_log) + np.exp(log_ham - max_log))
    prob_ham = np.exp(log_ham - max_log) / (np.exp(log_spam - max_log) + np.exp(log_ham - max_log))

    return log_spam > log_ham, prob_spam, prob_ham
```

Figure 4 : Code de la fonction prediction

Tests :

Avec les modifications demandées en fin de sujet (lissage), voici les résultats de la fonction test.

```
WHOLE', 'WHOM', 'WHOSE', 'WHY', 'WIDE', 'WIFE', 'WILL', 'WILLIAM', 'WINDOW', 'WISH',
'WITH', 'WITHIN', 'WITHOUT', 'WOMAN', 'WOMEN', 'WORD', 'WORDS', 'WORK', 'WORKED', 'WO
RKING', 'WORKS', 'WORLD', 'WOULD', 'WOULDNT', 'WRITING', 'WRITTEN', 'WRONG', 'WROTE',
 'YEAR', 'YEARS', 'YES', 'YET', 'YORK', 'YOU', 'YOUNG', 'YOUR']
apprentissage de bspam...
apprentissage de bham...
Probabilité à priori de SPAM : 0.16666666666666666
Probabilité à priori de HAM : 0.8333333333333334
Erreur de test sur 500 SPAM                    : 18 %
Erreur de test sur 500 HAM                     : 1 %
Erreur de test globale sur 1000 mails          : 10 %
```

Figure 5 : Résultats de la fonction test

Il est à noter que pour la réalisation de nos tests, en particulier celui-ci) on a paramétré la trace (l'affichage / débogage) et passée à false ici, c'est pourquoi les lignes de détails ne sont pas présentes. Cette fonctionnalité a tout de même été implémentée.
Voir figure 6.

```
HAM numéro 494 : P(Y=SPAM | X=x) = 4.667197750988291e-06, P(Y=HAM | X=x) = 0.99999533
28022491 ==> identifié comme un HAM
HAM numéro 495 : P(Y=SPAM | X=x) = 2.0956070928844337e-07, P(Y=HAM | X=x) = 0.9999997
904392908 ==> identifié comme un HAM
HAM numéro 496 : P(Y=SPAM | X=x) = 2.0346305362624873e-09, P(Y=HAM | X=x) = 0.9999999
979653695 ==> identifié comme un HAM
HAM numéro 497 : P(Y=SPAM | X=x) = 4.2070961501065084e-24, P(Y=HAM | X=x) = 1.0 ==> i
dentifié comme un HAM
HAM numéro 498 : P(Y=SPAM | X=x) = 2.965277317298031e-20, P(Y=HAM | X=x) = 1.0 ==> id
entifié comme un HAM
HAM numéro 499 : P(Y=SPAM | X=x) = 2.7571194717619107e-21, P(Y=HAM | X=x) = 1.0 ==> i
dentifié comme un HAM
```

Figure 6 : Extrait de la trace pour fonction test

Comparaisons :

Après la comparaison des tests avec et sans lissage, on observe que le lissage fait baisser d'un point l'erreur de test sur les SPAMs. Cela représente peut-être un faible gain mais le coût de modification étant moindre, on peut considérer rentable l'implémentation du lissage sur notre classifieur.
Pour l'apprentissage en ligne testé sur 1 mail lu en plus, on n'observe aucune modification significative. Ce résultat n'est pas surprenant, les effets et intérets de l'apprentissage en ligne se font ressentir avec une utilisation sur le long terme et des mises à jour régulière du classifieur.

```
SANS lissage - SPAM : 19%  |  HAM : 1%
AVEC lissage - SPAM : 18%  |  HAM : 1%
Vérification après import - SPAM : 18%  |  HAM : 1%
Avant mise à jour - mSpam : 500, mHam : 2500
Avant mise à jour - Pspam : 0.1667, Pham : 0.8333
Après mise à jour - mSpam : 501, mHam : 2501
Après mise à jour - Pspam : 0.1669, Pham : 0.8331
Après mise à jour - SPAM : 18%  |  HAM : 1%
```

Figure 7 : Résultats des comparaisons sur lissage et apprentissage en ligne

Pour tester dans de conditions plus réalistes, on peut se dire qu'on prend comme base 300 mails pour notre classifieur et que l'on met à jour en ligne pour les autres mails (jusqu'à 1200 mail pour les hams) de la base, un à un.
On met le tout dans un graphe pour visualiser les résultats.
Voir figure 8.

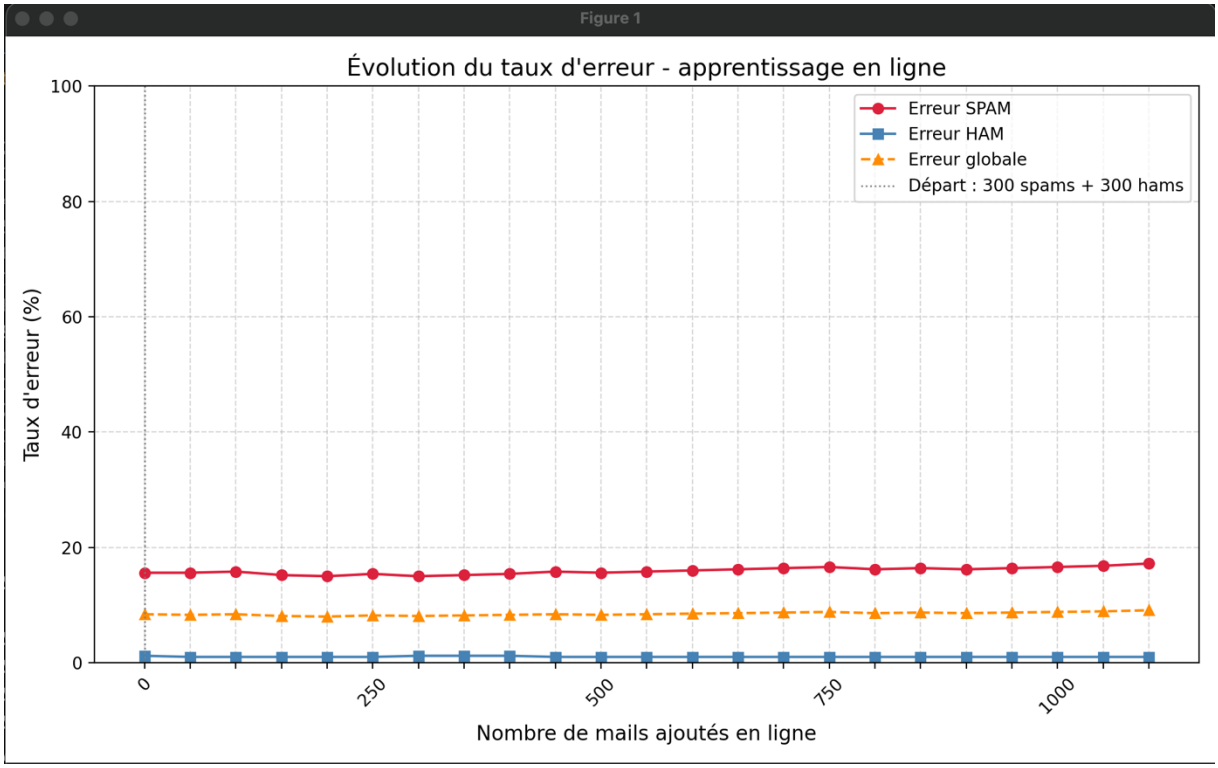

Figure 8 : Résutat apprentissage en ligne

Ces résultats sont surprenants puisqu'en principe l'erreur devrait varier et s'améliorer.
On en conclut que notre test n'est peut-être pas bien réalisé ou pas assez conséquent.